

Clase 06 (04/28/2026)

1. Argumentos en ROS2

- Conjunto de argumentos que permiten reconfigurar diversas propiedades de los nodos sin programación adicional
- Las opciones van delante de la opción `--ros-args` y solo se permite al iniciar el nodo
 - Cambio de nombre de nodo: `-r __node:=<nuevo_nombre_nodo>`
 - Cambio de *namespace*: `-r __ns:=<nombre_namespace>`
 - Cambio de nombre de *topic*: `-r <topic>:=<nuevo_nombre_topic>`
 - Nivel de log: `--log-level <info|debug|warn|error>`
 - Parámetros: `-p <nombre_parametro>:=<valor>`

2. Sistema *launch* de ROS2

- Un sistema ROS 2 puede estar compuesto de muchos nodos que se ejecutan en diversos procesos (e incluso en distintos dispositivos). Si bien es posible ejecutar manualmente cada uno de estos nodos, se vuelve engorroso rápidamente.
- El sistema de "lanzamiento" de ROS simplifica la ejecución de sistemas complejos
- También se encarga de monitorear el estado de los procesos lanzados, e informar y/o reaccionar ante cambios
- Los archivos que describen la configuración completa del sistema pueden ser escritos en XML, YAML o *Python*, y se ubican dentro de la carpeta `launch` dentro del paquete. Para compilarlos adecuadamente se debe modificar el archivo `setup.py`.
- Para mayor flexibilidad, se prefiere en *Python* porque así se tiene acceso a todas las funciones del lenguaje y las librerías (`ros2/launch` y `ros2/launch_ros`)

2.1. Archivos *launch* en *Python*

- Dentro del archivo se debe implementar la función `generate_launch_description()` que devuelve un objeto de tipo `launch.LaunchDescription()`
- El objeto `LaunchDescription` se compone de una colección o lista de "*acciones*". Para esto se utilizan dos librerías principales:
 - `launch`: Librería independiente de ROS2
 - `launch_ros`: Librería específica con acciones de ROS2

Acciones más utilizadas

■ Acciones

- Ejecutar un nodo (`launch_ros.actions.Node`)
- Ejecutar un comando (`launch.actions.ExecuteProcess`)
- Incluir otros archivos launch (`launch.actions.IncludeLaunchDescription`)
- Declarar argumentos (`launch.actions.DeclareLaunchArgument`)

■ Sustituciones

- Substituir por argumentos (`launch.substitutions.LaunchConfiguration`)
- Utilizar variables de entorno (`launch.substitutions.EnvironmentVariable`)
- Utilizar la salida de un comando (`launch.substitutions.Command`)
- Rutas de archivos (`launch.substitutions.PathJoinSubstitution`)
- Obtener la ubicación de un paquete de ROS (`launch_ros.substitutions.FindPackageShare`)

■ Condicionales

- Condición Si <verdadero> entonces (`launch.conditions.IfCondition` o `launch.conditions.LaunchCondition`)
- Condición Si <falso> entonces (`launch.conditions.UnlessCondition` o `launch.conditions.LaunchCondition`)

■ Eventos

- Encadenar ejecución (`launch.event_handlers.OnProcessExit`)

3. Laboratorio

Práctica integradora Unidad I *